QISKIT FALL FEST 2026

PHASE 1 — GIANT CHALLENGE KIT

20 Problem Statements • 5 Tracks • One Participant Guide

DOCUMENT PURPOSE
This document consolidates the Phase 1 challenge information that was previously distributed across multiple PDFs and challenge documents. Participants should use this document as the main navigation and operational guide, while the previously distributed PDFs remain available as supporting/reference material.

IMPORTANT PARTICIPANT UPDATE
Processor B is treated in this kit as the 9-qubit execution architecture communicated to participants. Processor C is a 13-qubit architecture for study/comparison only; participants are NOT required to implement or execute their solution on Processor C. This update is intended to prevent unnecessary rework.

NO-RESTART PRINCIPLE
Do not throw away work already completed under earlier QFF Phase 1 instructions. Keep your problem formulation, algorithm, experiments and results. Use the consolidated rules in this kit to complete or relabel the remaining benchmark/analysis work.

QFF 2026 Phase 1 Challenge Kit

1. Document Control & How to Use This Kit

1.1 What this document contains

- Universal Phase 1 rules and participant workflow.
- The consolidated architecture/benchmarking policy.
- All 20 challenge entries: 3 defined problem statements + 1 Open Innovation challenge for each of 5 tracks.
- Submission structure, common metrics, judging rubric and participant FAQ.
- A list of the previously distributed source PDFs/documents that should remain available.
- Organizer-only release checks at the end of the document.

1.2 Source and precedence

Source baseline: The problem-statement descriptions, motivations, baseline tasks, deliverables, metrics, data examples and difficulty estimates in this document are consolidated from the existing QFF Phase 1 challenge material, especially the QFF PS Executive Summary and related challenge documents.

Operational update: The participant-facing processor policy in this consolidated kit intentionally reflects the current organizer decision communicated for this release: Processor B = 9 qubits; Processor C = 13 qubits and study-only.

Legacy material: Previously distributed PDFs should remain accessible. They are retained for transparency and historical context; participants should not be asked to search across them to reconstruct the challenge.

1.3 Phase 1 in one page

1. Choose one track.
2. Choose one problem statement within that track: a defined PS or the track's Open Innovation option.
3. Read the corresponding challenge section in this kit.
4. Build the core solution and validate it against the specified classical/exact baseline where applicable.
5. Benchmark the solution on the required architecture target(s).
6. Analyze architecture effects such as qubit count, connectivity, CX count, circuit depth and SWAP/routing overhead where applicable.
7. Design and justify a custom Processor D (or Platform D for PQC) where the selected challenge requires it.
8. Prepare a reproducible repository with code, environment information, data, results and report.
9. Submit the repository link according to the Phase 1 submission instructions.

2. Universal Phase 1 Framework

2.1 Team and submission model

- Teams: 2–6 members.
- Phase 1 teams continue as the same teams into Phase 2.
- One team selects one problem statement.
- Submission is a GitHub repository or Hugging Face Space containing the required implementation and documentation.
- Code should be reproducible and should run end-to-end in the organizer-provided environment or a clearly specified compatible environment.

2.2 Common algorithm ↔ architecture philosophy

Phase 1 is not only an algorithm implementation exercise. Participants are expected to connect the algorithm to hardware architecture: connectivity, routing, circuit depth, two-qubit-gate cost, noise where relevant, and the trade-offs introduced by the processor model.

Important: Processor analysis should explain why results differ; it is not enough to submit three independent execution outputs.

2.3 Common architecture metrics

- Logical qubits / physical qubits used.
- Two-qubit gate (CX) count.
- Circuit depth.
- SWAP count or routing overhead where a routing stage is applicable.
- Single-qubit gate count where useful.
- Problem-specific performance metric such as energy error, objective value, accuracy, fidelity or approximation ratio.
- Noisy-vs-ideal comparison where the selected problem requires noise analysis.

2.4 Common judging rubric

Category	Weight	What is assessed
Solution correctness & performance	40	Correct implementation, meaningful results, comparison to a known/classical baseline where applicable, and effective

		use of the selected algorithm.
Architecture analysis	30	Quality of benchmarking/analysis, circuit resources, connectivity/routing effects, SWAP overhead, depth and interpretation.
Innovation & custom design	30	Quality and justification of Processor D / Platform D and whether it addresses an identified bottleneck.

Discretionary tie-breakers: clarity of report, quality of visualizations, code organization and documentation may help distinguish closely scored submissions.

3. Participant-Facing Processor Policy

THE THREE-ARCHITECTURE POLICY FOR THIS KIT

Processor A is the 5-qubit architecture. Processor B is the 9-qubit contemporary execution architecture used for the main participant benchmark. Processor C is a 13-qubit experimental/reference architecture used for architectural study only. Participants do not need to implement or execute their solution on Processor C.

3.1 Processor A — 5-qubit

- Physical qubits: 5.
- Topology: nearest-neighbor chain. Coupling map: 0–1–2–3–4 (bidirectional CX connectivity).
- Purpose: historical/baseline architecture and a compact connectivity case study.
- Use A when the selected circuit/problem fits its physical-qubit capacity.
- For a problem requiring more than 5 logical qubits, do not delete variables merely to force the full problem into A. Treat A as a capacity/routing reference and perform the full-size execution on Processor B where permitted.
- Expected architectural focus: limited nearest-neighbor connectivity and resulting routing/SWAP overhead.

3.2 Processor B — 9-qubit participant execution architecture

- Physical qubits: 9. This is the fixed participant-facing Processor B value for Phase 1.
- Role: primary contemporary architecture for executing the defined quantum problem instances.
- Topology: fixed 9-qubit heavy-hex-inspired sparse graph. Coupling map: 0–1, 1–2, 2–3, 2–4, 4–5, 5–6, 4–7, 7–8 (bidirectional CX connectivity).
- Participants must construct their Processor B model from the architecture specification and coupling map provided in this kit. The participant-created processor_B.json file must represent that specification for the required benchmark.
- Where a problem uses fewer than 9 logical qubits, participants may use the necessary subset while documenting the layout.
- Where a problem uses 6–8 logical qubits, B is the default full-size execution target.

3.3 Processor C — 13-qubit study/reference architecture

- Physical qubits: 13. This is the fixed participant-facing Processor C reference value for Phase 1.
- Role: architectural study/comparison only.
- Participants are NOT required to build their complete solution for Processor C.
- Reference topology: 12-node ring (0–1–2–3–4–5–6–7–8–9–10–11–0) plus central qubit 12 connected to 0, 2, 4, 6, 8 and 10. This topology is for architectural study only.
- No C implementation or C execution is required for the minimum Phase 1 submission unless a specific PS explicitly says otherwise in a future organizer update.

3.4 Gate and noise policy

Basis gates for the common quantum benchmark: {rz, sx, x, cx}. Measurement and reset operations are execution operations and are not included in the reported gate-cost comparison.

Fixed-noise model: single-qubit depolarizing probability = 0.001 per 1-qubit gate; CX depolarizing probability = 0.005 per CX; symmetric readout bit-flip probability = 0.01 per measured qubit.

The same synthetic noise parameters are used for comparable Processor A/B noisy experiments so that routing and architecture effects can be compared fairly.

Shots: 4096 for reported sampled results unless a problem statement explicitly specifies another count.

Transpilation baseline: Qiskit transpile with optimization_level=1, SABRE-based layout/routing, and transpiler seed 42. Teams may run additional compiler experiments, but the required benchmark must use this baseline.

Randomness: use seed 42 for organizer-defined baseline instances and parameter initialization wherever a seed is required; record any additional seeds used by the team.

For ideal-only problems, do not add noise merely to create extra work. For problem statements that explicitly study noise, use the fixed model above.

Processor C remains study/reference only and is not a required execution target.

3.5 Routing and compilation — participant rule

- Participants must build the coupling maps for Processor A and Processor B themselves based on the architecture specifications provided in this kit. Do not assume an organizer-provided coupling-map file.
- Report the logical circuit before mapping and the compiled/transpiled circuit after mapping.
- Report CX count, depth and routing/SWAP overhead.
- Do not manually alter the benchmark topology simply to improve a result.
- If a participant uses a different compiler/layout strategy as an experiment, label it clearly as an additional experiment rather than replacing the required benchmark configuration.

3.6 Compatibility rule for existing participant work

Existing participant work remains valid. If your implementation was built using an earlier QFF document, do not restart the project solely because this kit consolidates the instructions. Adapt only the minimum configuration needed to match the fixed Processor A/B benchmark and the 9-qubit Processor B definition.

DO NOT RESTART YOUR PROJECT

If you started using an earlier document, keep your existing problem formulation and core implementation. The 9-qubit B definition should be used for the main B benchmark going forward. C work is optional/study-only under this kit. Processor A work remains useful wherever the selected problem fits the 5-qubit capacity.

4. Reproducibility & Benchmark Protocol

Participants should construct their Processor A and Processor B models according to the processor specifications in this kit. They should use the released requirements.txt/environment specification for reproducibility rather than choosing incompatible package versions.

Item	Participant rule

Environment	Use the organizer-provided requirements/environment file in the final kit. Do not force participants to abandon a working environment unless a reproducibility issue is identified.
Processor B	9-qubit released coupling map is the authoritative benchmark topology.
Processor C	Study/reference only; no required implementation.
Shots	Use the shot count in the released benchmark configuration. If no separate file is supplied at release, 4096 shots is a practical proposed default.
Transpilation	Use the released Qiskit/transpiler settings. Report optimization level and routing/layout settings.
Seeds	Use fixed seeds for organizer benchmark runs; participants may add additional seeds for robustness.
Noise	Use the released fixed noise model when the PS requires noisy execution. Do not combine incompatible noise assumptions.
Comparison	Always preserve a classical/exact reference where the PS supplies one.

5. Submission Package

5.1 Recommended repository structure

```
project/
├── README.md
├── main.ipynb
├── requirements.txt
├── data/
│   ├── raw/
│   └── processed/
├── processors/
│   ├── processor_A.json
│   └── processor_B.json
├── src/
│   ├── __init__.py
│   ├── problem.py
│   ├── process.py
│   ├── routing.py
│   ├── metrics.py
│   └── visualization.py
├── results/
│   ├── tables/
│   └── figures/
├── report.md
└── assets/
    └── qff_phase1_latex_images/
```

```
├── eq_qubit_state.png
├── eq_bell_state.png
├── eq_graph.png
├── eq_bell_fidelity.png
├── eq_zz_expectation.png
└── eq_standard_error.png
```

Note: processor_A.json and processor_B.json are participant-created files. Completed Processor A/B implementations are not supplied by the organizers.

5.2 Minimum README contents

- Problem ID and title.
- Team members.
- One-paragraph problem summary.
- Approach/algorithm.
- Environment and execution instructions.
- Data source/instance description.
- Processor assumptions and coupling maps used.
- Results and architecture comparison.
- Processor D / Platform D design.
- Known limitations.

5.3 Recommended results table

```
problem_id, instance_id, architecture, logical_qubits, physical_qubits, objective_or_accuracy,
classical_reference, approximation_or_error, cx_count, depth, swap_count, shots, noise_model
```

6. Quantum Chemistry Track — VQE / Ab-Initio

Track focus: Use variational quantum algorithms to study molecular energies and hardware-efficient chemistry, while connecting ansatz/resource choices to architecture.

Participant workflow: Build the chemistry model → solve the reference problem → validate against an exact/classical result → benchmark the relevant circuit on the participant architecture → analyze resources → design Processor D.

PS-C1: Molecular Ground State Energy (H_2)

Problem: Compute the ground-state energy of H_2 in a minimal basis using VQE. The task introduces molecular Hamiltonian construction, variational ansätze and hardware constraints.

Baseline task: Use a standard hardware-efficient or UCCSD-style ansatz and compute the H_2 ground-state energy near the equilibrium bond length (~0.74 Å). Compare with the exact/classical reference; chemical accuracy (~0.0016 Hartree) is a useful sanity check.

Required deliverables: A Jupyter notebook containing Hamiltonian construction, ansatz, VQE execution, architecture/resource comparison, coupling-map diagrams, convergence plot where applicable, README and final summary.

Evaluation metrics: Energy error relative to the exact reference; CX count; circuit depth; SWAP/routing overhead; and, if noise is enabled, state fidelity or variance.

Data / instances: Organizer-provided 4×4 H_2 Hamiltonian/operator at R≈0.74 Å; optional nearby geometries.

Hints / guidance: Use Qiskit Nature or a precomputed operator. Focus on how ansatz depth and mapping affect accuracy and resource cost.

PS-C2: Diatomic Potential Energy Surface

Problem: Map the potential-energy curve of H_2 (or LiH) by computing the ground-state energy at multiple bond lengths.

Baseline task: Run VQE over approximately 5–7 bond distances. Reuse the same ansatz where possible so the comparison isolates the effect of geometry and hardware/resource requirements.

Required deliverables: Notebook that loops over bond distances, stores energies, plots the potential-energy curve, compares processors, and reports resource usage.

Evaluation metrics: Maximum energy deviation from exact values; ability to locate the correct minimum; CX/depth/SWAP trends across bond distance.

Data / instances: Hamiltonian data or generated molecular operators for a set of bond distances, e.g. H_2 stretching from roughly 0.4–2.0 Å.

Hints / guidance: Keep the ansatz strategy controlled across distances. Record optimizer convergence and preserve intermediate data.

PS-C3: Hardware-Efficient Chemistry

Problem: Target a somewhat larger molecule such as H_2O, HeH^+ or a short three-atom chain using reduction techniques so the quantum model remains small enough for NISQ-style simulation.

Baseline task: Construct a reduced Hamiltonian using methods such as frozen-core/active-space reduction, qubit tapering or symmetry reduction. Use a hardware-efficient, adaptive or VQE ansatz with no more than 8 qubits for the reduced model.

Required deliverables: Document every reduction step; provide the reduced Hamiltonian, ansatz, VQE results, architecture comparison, circuit diagrams and Processor D proposal.

Evaluation metrics: Ground-state energy error for the reduced model; qubits used; CX count; depth; SWAPs; optimizer convergence.

Data / instances: One organizer-provided reduced Hamiltonian, e.g. H_2O at equilibrium; optional integrals.

Hints / guidance: Explain why each reduction is scientifically reasonable. The challenge is not simply to make the circuit smaller; it is to preserve a meaningful physical problem.

C-OI: Open Innovation — Quantum Chemistry

Problem: Propose a quantum-chemistry problem beyond C1–C3, such as an excited-state calculation, molecular property estimation, reaction-related computation or geometry optimization.

Baseline task: Submit a 1–2 page proposal describing the scientific objective, expected quantum algorithm, data/model, scale and why the chosen problem is interesting or challenging classically. After approval, execute the approved task using the common Phase 1 architecture/Processor D workflow.

Required deliverables: Approved proposal, reproducible implementation, validation/reference method, architecture/resource analysis, Processor D design and final report.

Evaluation metrics: Scientific correctness, quality of the reference comparison, circuit/resource analysis and quality of the custom architecture.

Data / instances: Participant-proposed, subject to organizer approval.

Hints / guidance: Keep the scope small enough for Phase 1. The proposal must be concrete enough to define a reproducible benchmark.

7. Quantum Optimization Track — QAOA / QUBO

Track focus: Encode classical optimization problems into QUBO/Ising form, solve them variationally and study how sparse connectivity changes the cost of implementing the optimization circuit.

Participant workflow: Formulate the objective → establish classical optimum → build QAOA/variational circuit → run the feasible instance → compare quality and architecture cost → design Processor D.

PS-O1: Max-Cut on a Weighted Graph

Problem: Solve a small weighted Max-Cut problem using QAOA. The task tests how a combinatorial optimization problem maps to Z_iZ_j interactions and how connectivity affects routing.

Baseline task: Use a 6-vertex weighted graph. Formulate the Max-Cut QUBO/cost Hamiltonian and run QAOA at p=1 or p=2 with a classical optimizer. Report the best cut and weight.

Required deliverables: Notebook constructing the QUBO and QAOA circuit; runs/parameter restarts; best cut; classical optimum comparison; graph visualization; architecture analysis.

Evaluation metrics: Approximation ratio; probability of the optimal bitstring; CX count; circuit depth; SWAP/routing overhead.

Data / instances: Participants must construct or select a 6-vertex weighted graph, with integer edge weights in the range 1–5. The selected graph must remain fixed for all required Processor A, Processor B and Processor D comparisons. Participants may explore additional graphs experimentally.

Hints / guidance: The cost terms require two-qubit interactions. Sparse connectivity can therefore increase routing overhead. Keep the selected graph unchanged across the required Processor A, Processor B and Processor D comparisons.

PS-O2: Constrained Portfolio Optimization

Problem: Maximize a return-risk objective for a small portfolio selection problem encoded as a QUBO with a cardinality/budget constraint.

Baseline task: Use 4 assets with expected returns and covariance/risk information. Encode the exact-k selection constraint with a penalty term and run QAOA at p=1.

Required deliverables: QUBO matrix, QAOA circuit, classical optimum, quantum portfolio and objective, valid-solution frequency, architecture/resource comparison.

Evaluation metrics: Quantum objective / classical optimum ratio; frequency of valid solutions; CX count; depth; SWAPs.

Data / instances: Reference example: returns [5,3,4,2], 4×4 risk matrix, k=2 and penalty λ=2; organizers may replace/lock the exact matrix in the final data package.

Hints / guidance: The key modeling step is the penalty for the cardinality constraint. Explain how penalty strength changes feasibility versus objective quality.

PS-O3: Knapsack Optimization

Problem: Solve a 0/1 knapsack problem using a variational quantum approach, including an inequality constraint on total weight.

Baseline task: Use 5 items, maximize value subject to a weight capacity, convert the problem to a cost/QUBO form and use shallow QAOA (p=1).

Required deliverables: Problem formulation, circuit, classical brute-force comparison, selected items, payoff landscape for one benchmark architecture, and architecture comparison.

Evaluation metrics: Percentage of optimal value reached; probability of a valid solution; CX count; depth; SWAPs.

Data / instances: Reference example: values [10,4,7,5,8], weights [5,3,4,2,6], capacity 15.

Hints / guidance: Penalty tuning is central. A valid but lower-value solution is not equivalent to the true optimum; report both feasibility and objective quality.

O-OI: Open Innovation — Quantum Optimization

Problem: Define an optimization problem in a domain such as scheduling, vehicle routing, facility location, clustering or resource allocation.

Baseline task: Submit a 1–2 page proposal covering decision variables, objective, constraints, significance and a quantum mapping such as QAOA or VQE. After approval, follow the standard benchmark and custom-architecture workflow.

Required deliverables: Approved problem formulation, classical baseline, quantum formulation, implementation, architecture analysis, Processor D and final report.

Evaluation metrics: Correctness, objective quality, feasibility, architecture cost and custom-design justification.

Data / instances: Participant-proposed; organizer approval required.

Hints / guidance: Keep the decision space small enough for a reproducible Phase 1 experiment.

8. Quantum Simulation Track — Hamiltonian Simulation

Track focus: Study many-body Hamiltonians, real-time evolution and the interaction between Trotter depth, connectivity and noise.

Participant workflow: Construct Hamiltonian → establish exact reference → implement VQE or Trotter evolution → benchmark → analyze error/resource trade-offs → design Processor D.

PS-S1: Ising Model Ground State

Problem: Find the ground-state energy and magnetization of a 1D transverse-field Ising model (TFIM) chain.

Baseline task: Use a 4-qubit TFIM with J=1 and selected external fields g, e.g. 0.5, 1.0 and 1.5. Estimate the ground state using VQE (or the approved alternative) and compare against exact diagonalization.

Required deliverables: Hamiltonian construction, exact reference, variational circuit, results for each g, energy/magnetization comparison, architecture resource table.

Evaluation metrics: Energy error; magnetization error; CX count; depth; number of variational parameters; fidelity where available.

Data / instances: TFIM instances at J=1 with $g \in \{0.5, 1.0, 1.5\}$; organizer-provided Pauli operator/reference.

Hints / guidance: A layered ansatz with nearest-neighbor entanglers is a natural starting point. Explain how the chosen architecture supports the entangling pattern.

PS-S2: Real-Time Dynamics

Problem: Simulate real-time evolution of a small quantum system under the TFIM Hamiltonian and measure a time-dependent observable.

Baseline task: Use an initial state such as all spins up or a Néel state. Simulate evolution to a final time up to roughly $2\hbar/J$ using first-order Trotterization and compare against exact statevector evolution.

Required deliverables: Trotter-step circuit, explanation of step count, observable-vs-time comparison, exact reference, architecture resource table.

Evaluation metrics: Absolute observable error; gate count per Trotter step; total depth; SWAP count; error-versus-step trend.

Data / instances: Organizer-provided TFIM Hamiltonian and initial state; recommended step counts 1, 2 and 4.

Hints / guidance: ZZ interactions can be implemented with CNOT patterns. Compare the cost of increasing Trotter depth with the gain in approximation accuracy.

PS-S3: Trotter Error vs Noise

Problem: Study how digital-simulation noise changes the accuracy of a Trotterized TFIM circuit.

Baseline task: Reuse the PS-S2 setup at a fixed evolution time. Compare ideal and noisy executions using a fixed noise model and vary the number of Trotter steps.

Required deliverables: Noisy/ideal simulations, error-vs-depth plot, architecture comparison, noise/error table and interpretation of connectivity effects.

Evaluation metrics: Difference between ideal and noisy observable; fidelity loss where applicable; depth; CX count; SWAPs; noise accumulation versus Trotter depth.

Data / instances: Same Hamiltonian and Trotter instances as S2. The original material gives an illustrative example of ~0.5% CX and ~0.1% single-qubit error; the final released noise file is authoritative.

Hints / guidance: Do not mix noise assumptions across architectures. Keep the noise model fixed when the goal is to compare architecture-induced differences.

S-OI: Open Innovation — Quantum Simulation

Problem: Propose a small quantum-simulation scenario such as molecular real-time dynamics, a reaction model, a wavefunction simulation or a benchmark of a quantum linear-algebra subroutine.

Baseline task: Submit a short proposal defining the physical system, observable, initial state, why the problem is interesting/difficult classically and how it will be encoded.

Required deliverables: Approved proposal, model/Hamiltonian, implementation, reference solution, architecture analysis, Processor D and final report.

Evaluation metrics: Physical/model correctness, agreement with the reference, simulation resource cost, error analysis and custom architecture.

Data / instances: Participant-proposed; organizer approval required.

Hints / guidance: The proposal must specify enough information that two teams could independently reproduce the same benchmark.

9. Quantum Machine Learning Track — Variational QML

Track focus: Combine a trained quantum circuit with a classical learning objective and study both model quality and circuit-resource cost.

Participant workflow: Prepare data → establish classical baseline → build/train quantum model → evaluate → benchmark circuit cost → analyze robustness/noise → design Processor D.

PS-Q1: Biomedical Signal Classifier

Problem: Classify short ECG or EEG signal segments as normal versus abnormal using a hybrid quantum-classical variational classifier.

Baseline task: Extract 4–8 numerical features per segment, angle-encode them into a 4–8-qubit VQC, train circuit parameters with a classical optimizer and compare with logistic regression using the same features.

Required deliverables: Feature extraction, VQC construction/training, evaluation, per-architecture results, circuit-resource table, coupling-map visuals and Processor D design.

Evaluation metrics: Accuracy, F1 and recall, with recall treated as the primary metric; CX count; depth; SWAPs.

Data / instances: MIT-BIH Arrhythmia Database or a similar organizer-selected ECG/EEG source, reduced to 200–500 labeled samples with a fixed train/test split.

Hints / guidance: Normalize features appropriately before angle encoding. Use class weighting if imbalance materially affects recall.

PS-Q2: Quantum Kernel Classifier

Problem: Build a quantum kernel from circuit-state overlaps and use it inside a classical SVM, while testing the quantum model against tuned classical kernels.

Baseline task: Implement a feature map such as ZZFeatureMap, compute the quantum kernel, feed it to an SVM and compare against tuned RBF and polynomial kernels.

Required deliverables: Kernel construction, SVM results, multi-seed/statistical comparison, architecture resource table and Processor D proposal aimed at reducing kernel-evaluation cost.

Evaluation metrics: Accuracy/AUC with confidence intervals; CX count; depth; SWAPs; total circuit executions.

Data / instances: 2–3 datasets, including a standard benchmark subset and a synthetic dataset aligned with the chosen feature map, at roughly 4–8 qubits.

Hints / guidance: Tune the classical baselines honestly. Watch for kernel concentration and explain whether the quantum kernel remains informative.

PS-Q3: Noise-Resilient Variational Classifier

Problem: Study how a classifier trained ideally degrades under device noise and whether error-mitigation techniques recover useful performance.

Baseline task: Train the VQC in an ideal setting, evaluate on the participant execution architecture with the fixed noise model and apply at least two mitigation methods where feasible, such as zero-noise extrapolation and readout correction.

Required deliverables: Ideal/noisy/mitigated results, mitigation implementation, extra-execution cost table, variance/repeated-run analysis and Processor D focused on noise resilience.

Evaluation metrics: Accuracy in ideal/noisy/mitigated settings; total circuit executions/shots; variance; architecture cost.

Data / instances: Reuse the dataset/model from Q1 or Q2 where appropriate. The original material proposes Qiskit-style noise models mapped to the benchmark architectures.

Hints / guidance: A mitigation method is not successful merely because the output changes. Quantify whether it actually recovers the target metric and what extra sampling cost it introduces.

Q-OI: Open Innovation — Quantum Machine Learning

Problem: Propose a QML application such as fraud detection, quantum generative modeling or a materials-property predictor.

Baseline task: Submit a 1–2 page proposal covering dataset, quantum model, classical baseline, evaluation metrics and why the task is non-trivial at the chosen scale. After approval, follow the common architecture and Processor D workflow.

Required deliverables: Approved proposal, dataset preparation, quantum model, classical comparison, architecture analysis, Processor D and final report.

Evaluation metrics: Task-specific ML quality plus circuit/resource metrics and robustness.

Data / instances: Participant-proposed; organizer approval required.

Hints / guidance: A good proposal includes a meaningful classical baseline and a dataset small enough to make the quantum experiment reproducible.

10. Post-Quantum Cryptography Track — Quantum-Safe Systems

Track focus: This track is classical software rather than quantum-circuit execution. The fixed comparison points are Target Platforms A/B/C, followed by a custom Platform D.

Participant workflow: Build/measure the cryptographic system → validate correctness/security → benchmark on fixed target platforms → analyze performance/resource trade-offs → design Platform D.

IMPORTANT: PQC DOES NOT USE QUANTUM PROCESSORS
For this track, the A/B/C/D labels refer to target computing platforms, not quantum processor models. This preserves the Phase 1 algorithm ↔ architecture comparison idea without pretending that PQC implementations run as quantum circuits.

PS-P1: Quantum-Safe Secure Channel

Problem: Build a messaging/file-transfer demo using standardized post-quantum primitives: ML-KEM for key establishment and ML-DSA for authentication, followed by symmetric encryption.

Baseline task: Implement a handshake using a vetted library, derive a shared secret with ML-KEM, authenticate with ML-DSA, derive the symmetric key with HKDF and encrypt payloads using AES-256-GCM. Benchmark on Target Platforms A/B/C.

Required deliverables: Working demo, handshake flow diagram, timing/memory benchmark, explanation of the man-in-the-middle threat, and custom Platform D benchmark.

Evaluation metrics: Handshake correctness/interoperability; time; memory; code size; bytes exchanged.

Data / instances: Organizer-provided payloads of approximately 1 KB, 100 KB and 10 MB.

Hints / guidance: Do not implement cryptographic primitives yourself. Use a vetted library and document the exact parameter sets.

PS-P2: Crypto Inventory & Migration Planner

Problem: Build a scanner that finds classical cryptographic usage in source code, rates quantum risk and suggests post-quantum replacements.

Baseline task: Statically scan a provided multi-language sample repository, detect cryptographic API usage, classify risk and map findings to suggested replacements such as ML-KEM and ML-DSA.

Required deliverables: Scanner, risk report, precision/recall against organizer ground truth, benchmark on target platforms and migration recommendations.

Evaluation metrics: Detection precision/recall; scan time; memory usage.

Data / instances: Organizer-provided Python/Java/C sample repository with planted cryptographic usages; ground truth is withheld until judging.

Hints / guidance: Regex can be a starting point; AST-aware or rule-based analysis should reduce false positives and improve context awareness.

PS-P3: PQC Performance Benchmark Across Constrained Platforms

Problem: Compare ML-KEM, ML-DSA and SLH-DSA against classical RSA/ECC across resource-rich, constrained and embedded-style targets.

Baseline task: Measure key/ciphertext/signature sizes, operation time and memory for at least one fixed parameter set per algorithm, with at least 100 repetitions per measurement.

Required deliverables: Reproducible benchmark scripts, comparison tables, at least three charts, written trade-off conclusion and a custom Platform D stress test.

Evaluation metrics: Size in bytes; time in milliseconds; memory in KB; degradation ratio between Platform C and Platform A.

Data / instances: Organizer-fixed parameter sets, including examples such as ML-KEM-768, ML-DSA-65 and SLH-DSA-128s against RSA-2048 and P-256.

Hints / guidance: Use stable benchmarking methodology. Emulated targets should be measured consistently rather than using naive wall-clock timing alone.

P-OI: Open Innovation — Post-Quantum Cryptography

Problem: Propose a PQC-related systems/security challenge, such as a hybrid TLS 1.3 handshake under network constraints, timing-side-channel analysis, or harvest-now-decrypt-later migration-risk modeling.

Baseline task: Submit a 1–2 page proposal describing the system, threat model, algorithm(s), evaluation plan and why classical-only practice is insufficient. After approval, follow the standard Platform A/B/C/D workflow.

Required deliverables: Approved proposal, reproducible implementation/analysis, benchmark, security/threat-model discussion and Platform D.

Evaluation metrics: Security/correctness, benchmark quality, threat-model rigor and custom-platform justification.

Data / instances: Participant-proposed; organizer approval required.

Hints / guidance: Avoid proposing a topic that cannot be tested or compared objectively within Phase 1.

11. Custom Processor D / Platform D

11.1 Quantum tracks — Processor D

- Maximum of 12 qubits under the original framework.
- Connectivity may be any graph, planar or non-planar, provided it is explicitly defined.

- Gate set should remain within the allowed CX + single-qubit-rotation family.
- Nominal gate-error assumptions should not be more favorable than the organizer benchmark without justification.
- The design must address a bottleneck identified in A/B analysis, such as routing, depth, parallelization or noise exposure.
- Provide a reproducible coupling map and show the same problem re-run on D.

11.2 PQC track — Platform D

- Choose a specific resource budget or parameter-set/platform configuration that exposes a genuine trade-off.
- Re-run the benchmark on D.
- Explain why D is useful and what trade-off it creates rather than simply making the machine more powerful.

12. Participant FAQ

Question	Answer
Do I need to read all the old PDFs?	No. This Challenge Kit is intended to consolidate the operational information. The old PDFs remain available as reference documents.
Has Processor B changed?	For this participant-facing kit, use Processor B as a 9-qubit architecture. Do not revert to an older 7-qubit description.
How many qubits is Processor C?	13 qubits in the current kit. C is for architectural study/comparison only; participants do not need to execute their full solution on C.
Do I need to redo my project?	No. Keep your existing core implementation and results. Use the consolidated rules to complete the required benchmark/analysis.
What if my problem needs more than 5 qubits?	Do not remove variables solely to fit Processor A. Use A as a capacity/routing reference and use the 9-qubit B architecture for the full-size benchmark where the PS permits it.
Do I need Processor D?	Yes for the quantum problem statements unless the organizer publishes a PS-specific exception. PQC participants design Platform D.
Can I use my own graph/dataset?	Only where the PS explicitly permits it. Official benchmark instances must remain unchanged for the required comparison.
Can I use a different optimizer?	Yes for exploration unless the PS specifies a fixed method. Clearly report the optimizer and parameters used.
Do I need noisy simulation everywhere?	Only where the PS requires it or the released benchmark protocol specifies it. Do not invent inconsistent noise assumptions.
What should I report for architecture?	At minimum, report qubits, CX count, circuit depth and SWAP/routing overhead where applicable, plus the problem-specific quality metric.

13. Participant Closing Checklist

10. I selected one track and one problem statement.
11. I used the official instance/data for the required benchmark.

12. My classical/exact reference is documented where applicable.
13. My quantum implementation runs end-to-end.
14. I used the 9-qubit Processor B definition for the participant benchmark where applicable.
15. I did not treat Processor C as a required execution target.
16. I reported circuit depth, CX count and routing/SWAP overhead where applicable.
17. I included the required problem-specific metric.
18. I included Processor D / Platform D and explained why it addresses a bottleneck.
19. My repository contains code, environment information, data, results, README and report.
20. Another person can reproduce my main result from the repository instructions.